

{EPITECH}

HOMEMIND

BOOTSTRAP_

< GUIDE DE DÉMARRAGE STEP BY STEP />

BOOTSTRAP GUIDE



Ce guide t'accompagne étape par étape depuis zéro jusqu'à un robot fonctionnel avec IA heuristique. Chaque step a un **checkpoint** — ne passe à la suivante qu'une fois validé.

Prérequis

Catégorie	Ce qu'il te faut	Pourquoi
OS	Ubuntu 22.04 LTS (dual-boot ou VM)	ROS 2 supporte officiellement Ubuntu
Python	Python 3.10+ (inclus Ubuntu 22)	Langage principal du projet
Git	Git + compte GitHub ou GitLab	Versionner chaque étape
Hardware	Raspberry Pi 5, L298N, HC-SR04, servo	Phase hardware
3D	Accès imprimante 3D + FreeCAD	Chassis du robot

Phase 0 — Bootstrap hardware

1

Installer Ubuntu 22.04 LTS

Télécharge l'ISO depuis ubuntu.com et installe-le. Dual-boot recommandé si tu es sur Windows. Une VM VirtualBox fonctionne aussi pour commencer.

```
# Vérifie la version après installation lsb_release -a
```

✓ **Checkpoint : tu vois 'Ubuntu 22.04' dans le terminal**

2

Installer ROS 2 Humble

ROS 2 Humble est la version LTS recommandée. Suit les commandes dans l'ordre exact — une erreur de clé GPG est la cause d'échec la plus fréquente.

```
sudo apt update && sudo apt install -y curl sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \ -o
/usr/share/keyrings/ros-archive-keyring.gpg echo 'deb
[signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu jammy main' \ | sudo tee
/etc/apt/sources.list.d/ros2.list sudo apt update && sudo apt install -y
ros-humble-desktop
```

```
# Ajoute au .bashrc pour sourcer automatiquement echo 'source /opt/ros/humble/setup.bash'
>> ~/.bashrc && source ~/.bashrc
```

✓ **Checkpoint : ros2 --version affiche 'ros2 humble'**

3

Créer ton workspace ROS 2

Un workspace ROS 2 est un dossier structuré contenant tous tes packages. Il se crée une seule fois.

```
mkdir -p ~/robot_ws/src && cd ~/robot_ws colcon build source install/setup.bash echo 'source ~/robot_ws/install/setup.bash' >> ~/.bashrc
```

robot_ws/src/ → tes packages Python/C++

robot_ws/install/ → fichiers compilés (généré automatiquement)

✓ **Checkpoint : colcon build termine sans erreur**

4

Créer ton premier package ROS 2

Un package ROS 2 est l'unité de code de base. Crée-en un pour le contrôle moteur.

```
cd ~/robot_ws/src ros2 pkg create --build-type ament_python motor_controller cd ~/robot_ws && colcon build && source install/setup.bash
```

```
# Teste que le package existe ros2 pkg list | grep motor_controller
```

✓ **Checkpoint : le package apparaît dans ros2 pkg list**

5

Câbler les moteurs — L298N

Le L298N est un driver à pont en H pour contrôler deux moteurs DC avec direction et vitesse PWM.

ENA → GPIO 12 (PWM) | IN1 → GPIO 20 | IN2 → GPIO 21

ENB → GPIO 13 (PWM) | IN3 → GPIO 19 | IN4 → GPIO 26

12V moteur → alimentation séparée | GND → GND commun RPi

■ **Ne connecte JAMAIS le 12V moteur directement au RPi — alimente le L298N séparément.**

✓ **Checkpoint : les moteurs tournent dans les deux sens selon IN1/IN2**

6

Premier node ROS 2 — bouger les roues

Écris ton premier node ROS 2 qui souscrit à /cmd_vel et commande les GPIO en conséquence.

```
# motor_controller/motor_controller/move_node.py import rclpy from rclpy.node import Node
from geometry_msgs.msg import Twist import RPi.GPIO as GPIO class MoveNode(Node): def
__init__(self): super().__init__('move_node') self.sub = self.create_subscription( Twist,
'/cmd_vel', self.on_cmd, 10) GPIO.setmode(GPIO.BCM) GPIO.setup([20,21,19,26,12,13],
GPIO.OUT) def on_cmd(self, msg): fwd = msg.linear.x > 0 GPIO.output(20, fwd)
GPIO.output(21, not fwd) def main(): rclpy.init() rclpy.spin(MoveNode()) if __name__ ==
'__main__': main()
```

✓ **Checkpoint : ros2 topic pub /cmd_vel geometry_msgs/Twist fait bouger les roues**

Câbler les capteurs ultrason HC-SR04

Le HC-SR04 mesure la distance par ultrasons. Câble 3 capteurs minimum : avant, gauche, droite.

VCC → 5V | GND → GND | TRIG → GPIO 23 | ECHO → diviseur de tension → GPIO 24

■ Le ECHO sort du 5V — utilise un diviseur résistif ($1k\Omega + 2k\Omega$) pour protéger le GPIO 3.3V du RPi.

```
import RPi.GPIO as GPIO, time
GPIO.setmode(GPIO.BCM)
TRIG, ECHO = 23, 24
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.output(TRIG, True)
time.sleep(0.00001)
GPIO.output(TRIG, False)
while GPIO.input(ECHO) == 0:
    start = time.time()
while GPIO.input(ECHO) == 1:
    stop = time.time()
print(f'Distance: {(stop-start)*17150:.1f} cm')
```

✓ **Checkpoint : distance affichée cohérente avec un objet placé devant**

Imprimer le châssis en 3D

Utilise un design existant sur [Printables.com](https://www.printables.com) ('robot 2WD chassis') ou modélise le tien dans FreeCAD.

Contraintes à respecter :

Épaisseur minimale des parois : 2.5 mm

Encoches pour vis M3 dans les supports moteur

Espace batterie LiPo en position basse (centre de gravité bas)

Entretoise RPi : vis M2.5, espacement 58 × 49 mm

```
# Paramètres Cura recommandés #
Infill : 20% | Layer height : 0.2mm # Support : oui si
porte-à-faux > 45° | Matériau : PLA
```

✓ **Checkpoint : le châssis imprimé tient debout, les moteurs s'y fixent sans jeu**

Phase 1 — IA heuristique

9

Implémenter la FSM (Finite State Machine)

Une FSM est un automate à états finis. Le robot a 3 états : EXPLORE, AVOID, STOP. La transition dépend des lectures ultrason.

```
from enum import Enum class State(Enum): EXPLORE = 'explore' AVOID = 'avoid' STOP = 'stop'
class RobotFSM: def __init__(self): self.state = State.EXPLORE def update(self,
dist_front, dist_left, dist_right): if dist_front < 20: # obstacle devant – tourne du côté
le plus libre self.state = State.AVOID elif dist_front > 40: self.state = State.EXPLORE
return self.state
```

✓ Checkpoint : le robot change d'état quand tu approches la main

10

Implémenter le pathfinding A*

A* trouve le chemin optimal entre deux points. Ton salon devient une grille 2D où 0=libre, 1=obstacle.

```
import heapq def astar(grid, start, goal): open_set = [(0, start)] came_from = {} g_score =
{start: 0} while open_set: _, current = heapq.heappop(open_set) if current == goal: path =
[] while current in came_from: path.append(current) current = came_from[current] return
path[::-1] for dx, dy in [(0,1),(0,-1),(1,0),(-1,0)]: nb = (current[0]+dx, current[1]+dy)
if (0 <= nb[0] < len(grid) and 0 <= nb[1] < len(grid[0]) and grid[nb[0]][nb[1]] == 0):
new_g = g_score[current] + 1 if new_g < g_score.get(nb, float('inf')): came_from[nb] =
current g_score[nb] = new_g h = abs(nb[0]-goal[0]) + abs(nb[1]-goal[1])
heapq.heappush(open_set, (new_g+h, nb)) return []
```

$f(n) = g(n) + h(n)$ — g = coût réel parcouru, h = heuristique Manhattan

✓ Checkpoint : `astar(grid, (0,0), (5,5))` retourne un chemin valide

11

Logger les décisions en temps réel

Logger chaque décision permet d'analyser le comportement et de détecter les bugs de FSM.

```
import csv, time class DecisionLogger: def __init__(self, path='decisions.csv'): self.f =
open(path, 'w', newline='') self.w = csv.writer(self.f)
self.w.writerow(['timestamp','state','dist_front', 'dist_left','dist_right','action']) def
log(self, state, dists, action): self.w.writerow([ time.time(), state.value, dists[0],
dists[1], dists[2], action ]) self.f.flush() # écrit immédiatement sur disque def
close(self): self.f.close()
```

✓ Checkpoint : `decisions.csv` se remplit en temps réel pendant la navigation

Et après ? — Phases suivantes

Les phases 2, 3 et 4 (RL, Vision, LLM) sont décrites en détail dans le sujet principal. Ce guide t'a amené jusqu'à un robot fonctionnel avec IA heuristique — la base solide sur laquelle tout le reste s'appuie.

Phase	Première commande à taper	Durée
Phase 2 — RL	<code>pip install gymnasium stable-baselines3 torch</code>	~6 sem.
Phase 3 — Vision	<code>pip install ultralytics opencv-python</code>	~4 sem.
Phase 4 — LLM	<code>curl https://ollama.ai/install.sh sh</code>	bonus

Ressources utiles

docs.ros.org/en/humble — Documentation officielle ROS 2 Humble

stable-baselines3.readthedocs.io — Guide SB3 — PPO, SAC, TD3 expliqués

docs.ultralytics.com — YOLOv8 — démarrage en 5 minutes

ollama.ai — Ollama — faire tourner des LLMs en local

printables.com — Designs de châssis 3D open source

husarion.com/tutorials — Tutoriels ROS 2 + robotique pratique



Versionne ton code à chaque étape avec Git. Un `git commit` après chaque checkpoint te permettra de revenir en arrière sans stress.



Ce guide couvre les Phases 0 et 1. Réfère-toi au sujet principal **HomeMind Robot** pour les phases suivantes.